

TALLER PREVIO 1. PRÁCTICA GUIADA EN MONGODB

Creación, consulta, actualización, borrado, respaldo y restauración de una base de datos documental con MongoDB

Programa: Tecnólogo en Análisis y Desarrollo de Software — ADSO

Competencia: Modelado de los artefactos del software

Resultado de aprendizaje asociado: Estructurar el modelo de datos del software de acuerdo con las especificaciones del análisis.

Tipo de trabajo: Individual

Duración principal: 12 horas

Distribución: 2 sesiones de 6 horas cada una

Sesión adicional opcional: 1 sesión de 6 horas para refuerzo, restauración, ajustes y sustentación ampliada

Producto final: Base de datos funcional en MongoDB, archivo docker-compose.yml, script de comandos MongoDB, consultas ejecutadas, copia de seguridad, restauración comprobada y video individual explicando el paso a paso realizado.

1. Nota previa para el aprendiz

Este taller se realizará en el ambiente de formación, donde el instructor orientará la práctica con **MongoDB ejecutado desde Docker Compose**.

La instalación directa de MongoDB en el computador no hará parte del desarrollo central del taller. En su lugar, se usará Docker Compose para levantar el servicio de MongoDB de manera controlada, práctica y repetible.

El propósito es que cada aprendiz comprenda cómo se estructura una base de datos documental, cómo se insertan documentos tipo JSON, cómo se consultan, actualizan, eliminan, respaldan y restauran datos usando MongoDB.

Docker Compose permite definir servicios, redes y volúmenes en un archivo de configuración, lo que facilita levantar entornos de trabajo de forma organizada.

2. Propósito del taller

Crear y administrar una base de datos en MongoDB de manera guiada, comprendiendo cómo se organizan los datos mediante **bases de datos, colecciones, documentos, campos, objetos JSON, arreglos y documentos embebidos**.

El aprendiz desarrollará una práctica basada en la tienda tecnológica `tienda_adso`, usando los mismos datos del taller previo en MySQL: clientes, productos, pedidos y detalle de pedido. Sin embargo, en esta versión se explicará cómo estos datos pueden representarse en MongoDB mediante documentos tipo JSON.

MongoDB almacena los datos en documentos BSON, que son una representación binaria de documentos similares a JSON, con soporte para más tipos de datos.

3. Resultado esperado

Al finalizar el taller, cada aprendiz estará en capacidad de:

1. Comprender qué es MongoDB.
2. Diferenciar una base de datos relacional de una base de datos documental.
3. Comprender qué es un objeto JSON.
4. Identificar la estructura básica de un documento en MongoDB.
5. Levantar MongoDB usando Docker Compose.
6. Ingresar a MongoDB usando `mongosh`.
7. Crear una base de datos llamada `tienda_adso`.
8. Crear colecciones para clientes, productos y pedidos.
9. Insertar documentos en colecciones.
10. Consultar documentos usando `find`.
11. Aplicar filtros en consultas.
12. Ordenar resultados.
13. Consultar documentos relacionados mediante referencias y agregaciones.
14. Usar documentos embebidos para representar el detalle de un pedido.
15. Actualizar documentos con `updateOne` y `updateMany`.
16. Eliminar documentos de forma controlada.
17. Eliminar colecciones y bases de datos.
18. Realizar copia de seguridad con `mongodump`.
19. Restaurar la base de datos con `mongorestore`.
20. Sustentar el proceso mediante un video individual con explicación paso a paso.

Las operaciones CRUD en MongoDB corresponden a crear, leer, actualizar y eliminar documentos; estas se realizan con métodos como `insertOne`, `insertMany`, `find`, `updateOne`, `updateMany`, `deleteOne` y `deleteMany`.

4. Sustento metodológico

Este taller se desarrolla bajo la estrategia de **aprender haciendo**, propia de la formación por competencias. El aprendiz no solo observa comandos, sino que los ejecuta, verifica resultados, corrige errores y comprende cómo las decisiones del diseño de datos afectan el funcionamiento de un sistema.

La actividad es individual porque cada aprendiz debe vivir el proceso completo: levantar el servicio, crear la base de datos, insertar documentos, consultar, actualizar, eliminar, respaldar, restaurar y explicar lo realizado mediante un video.

El instructor acompaña el proceso con orientación paso a paso, preguntas mediadoras y retroalimentación técnica.

5. Sustento técnico

En una base de datos relacional como MySQL, los datos se organizan en tablas, filas, columnas, claves primarias y claves foráneas. El taller base trabaja justamente esos conceptos para comprender cómo se relacionan las tablas.

En MongoDB, el enfoque cambia. Ya no se trabaja principalmente con tablas, sino con documentos almacenados dentro de colecciones.

5.1 Conceptos principales de MongoDB

Elemento técnico	Explicación sencilla
Base de datos	Contenedor principal donde se guardan colecciones.
Colección	Agrupación de documentos. Es parecida a una tabla, pero más flexible.
Documento	Registro individual en MongoDB. Se parece a un objeto JSON.
Campo	Nombre de un dato dentro del documento.
Valor	Información almacenada en un campo.
<code>_id</code>	Identificador único de cada documento. Cumple un papel parecido a una clave primaria.
Objeto JSON	Estructura de datos formada por pares clave-valor.
Arreglo	Lista de valores o documentos dentro de

Elemento técnico	Explicación sencilla
	un documento.
Documento embebido	Documento dentro de otro documento.
Referencia	Campo que guarda el identificador de otro documento.
Consulta	Instrucción para buscar documentos.
Agregación	Proceso para transformar, agrupar o combinar datos.
\$lookup	Operación de agregación que permite combinar información de colecciones.
Copia de seguridad	Respaldo de la base de datos para poder recuperarla.
Restauración	Proceso de recuperar una base de datos desde un respaldo.

6. ¿Qué es un objeto JSON?

JSON significa **JavaScript Object Notation**. Es un formato sencillo para representar datos mediante pares de clave y valor.

Un objeto JSON se escribe entre llaves { }.

Ejemplo:

```
{
  "nombre": "Ana Pérez",
  "correo": "ana.perez@correo.com",
  "telefono": "3001112233"
}
```

En este ejemplo:

- nombre es una clave o campo.
- "Ana Pérez" es el valor.
- correo es una clave.
- "ana.perez@correo.com" es el valor.
- telefono es una clave.
- "3001112233" es el valor.

6.1 Tipos de valores en JSON

Un objeto JSON puede guardar diferentes tipos de valores:

```
{
  "nombre_producto": "Mouse inalámbrico",
  "precio": 45000,
  "stock": 20,
  "activo": true,
  "categorias": ["Tecnología", "Accesorios"],
  "fabricante": {
    "nombre": "Tech Sena",
    "pais": "Colombia"
  }
}
```

En este ejemplo se observan:

Tipo	Ejemplo
Texto	"Mouse inalámbrico"
Número	45000
Booleano	true
Arreglo	["Tecnología", "Accesorios"]
Objeto embebido	"fabricante": { "nombre": "Tech Sena" }

6.2 Reglas básicas para escribir JSON

1. Las claves normalmente van entre comillas dobles.
2. Los textos van entre comillas dobles.
3. Los números no llevan comillas.
4. Los valores booleanos se escriben como true o false.
5. Los objetos usan llaves { }.
6. Los arreglos usan corchetes [].
7. Cada par clave-valor se separa con coma.
8. No se debe poner coma después del último campo.

6.3 JSON y MongoDB

MongoDB trabaja con documentos parecidos a JSON. Técnicamente, MongoDB almacena los documentos como BSON, que es una representación binaria de JSON con tipos adicionales.

Para los aprendices, puede entenderse así:

JSON: forma fácil de escribir y leer los datos.

BSON: forma interna optimizada que usa MongoDB para almacenar esos datos.

7. Materiales necesarios para el ambiente de formación

- Computador individual por aprendiz.
- Docker Desktop instalado o Docker Engine disponible.
- Acceso a consola, terminal, CMD, PowerShell o terminal integrada de Visual Studio Code.
- Visual Studio Code.
- Carpeta digital para evidencias.
- Guía del taller.
- Lista de chequeo.
- Video beam o pantalla para demostraciones del instructor.
- Acceso a internet.
- Espacio en disco suficiente para contenedores, volúmenes y respaldo.
- Herramienta para grabar pantalla y audio.
- Cuenta o medio para entregar el video: enlace de Drive, OneDrive, YouTube no listado o plataforma indicada por el instructor.

8. Escenario de práctica

Cada aprendiz construirá una base de datos documental para una tienda tecnológica sencilla llamada:

tienda_adso

La tienda necesita registrar:

- Clientes.
- Productos.
- Pedidos.
- Detalle de los productos incluidos en cada pedido.

El taller base de MySQL trabaja cuatro tablas: clientes, productos, pedidos y detalle_pedido. En MongoDB se conservarán los mismos datos, pero se representarán mediante colecciones y documentos.

9. Modelo de datos de la práctica en MongoDB

9.1 Colecciones del sistema

Colección	¿Qué almacena?
clientes	Información de las personas que realizan pedidos.
productos	Información de los productos disponibles para la venta.
pedidos	Información general de cada pedido realizado por un cliente y el detalle de productos incluidos.

9.2 Diferencia importante frente a MySQL

En MySQL, el detalle del pedido se guarda en una tabla separada llamada `detalle_pedido`.

En MongoDB, para este taller, el detalle del pedido se guardará como un **arreglo de objetos JSON dentro del documento pedido**.

Ejemplo:

```
{
  _id: 1,
  id_cliente: 1,
  fecha_pedido: new Date(),
  estado: "CREADO",
  total: 225000,
  detalle_pedido: [
    {
      id_producto: 1,
      cantidad: 1,
      precio_unitario: 45000
    },
    {
      id_producto: 2,
      cantidad: 1,
      precio_unitario: 180000
    }
  ]
}
```

9.3 Explicación para el aprendiz

Un cliente puede realizar muchos pedidos.

Cada pedido pertenece a un cliente, por eso el documento pedido guarda el campo `id_cliente`.

Un pedido puede incluir varios productos. En MongoDB, esos productos pueden guardarse dentro del mismo documento pedido mediante un arreglo llamado `detalle_pedido`.

Cada elemento del arreglo `detalle_pedido` es un objeto JSON que indica:

- Producto comprado.
- Cantidad.
- Precio unitario.

En MongoDB no se trabaja con claves foráneas obligatorias como en MySQL. La relación se puede representar mediante referencias, pero la base de datos no impide automáticamente que se elimine un cliente que tenga pedidos. Por eso el desarrollador debe validar cuidadosamente la integridad de los datos desde el diseño y desde la aplicación.

10. Distribución del tiempo

Sesión 1 — 6 horas

Momento	Actividad	Tiempo
1	Presentación del taller, propósito y producto esperado	20 min
2	Reconocimiento de conceptos: MongoDB, colección, documento, campo, <code>_id</code> , JSON y BSON	45 min
3	Explicación de Docker Compose para levantar MongoDB	45 min
4	Creación del archivo <code>docker-compose.yml</code> y verificación del entorno	40 min
5	Ingreso a MongoDB con <code>mongosh</code> y creación de la base de datos	40 min
6	Inserción guiada de clientes y productos	80 min

Momento	Actividad	Tiempo
7	Consultas básicas con find, filtros y ordenamiento	70 min
8	Cierre de sesión y verificación de evidencias	20 min
Total		360 min

Sesión 2 — 6 horas

Momento	Actividad	Tiempo
1	Repaso de la sesión anterior	20 min
2	Inserción de pedidos con detalle embebido	50 min
3	Consultas relacionadas usando referencias, arreglos y agregaciones	80 min
4	Actualización de documentos con updateOne y updateMany	35 min
5	Eliminación controlada de documentos y colecciones	35 min
6	Copia de seguridad con mongodump	35 min
7	Borrado controlado y restauración con mongorestore	50 min
8	Organización de evidencias y preparación del video individual	45 min
9	Cierre y verificación final	10 min
Total		360 min

Sesión 3 opcional — 6 horas

Momento	Actividad	Tiempo
1	Corrección de errores en comandos y scripts	60 min
2	Refuerzo de JSON, arreglos y documentos embebidos	70 min
3	Refuerzo de consultas con filtros y agregaciones	90 min
4	Repetición de backup, borrado y restauración	80 min
5	Organización de evidencias	30 min
6	Revisión de videos individuales	30 min

Momento	Actividad	Tiempo
Total		360 min

11. Comandos y operaciones que se practicarán

Categoría	Comando u operación	Uso en el taller
Docker	<code>docker compose up -d</code>	Levantar MongoDB en segundo plano.
Docker	<code>docker compose ps</code>	Verificar contenedores activos.
Docker	<code>docker compose logs</code>	Revisar mensajes del contenedor.
Docker	<code>docker compose down</code>	Detener y eliminar contenedores.
Docker	<code>docker compose down -v</code>	Eliminar contenedores y volúmenes.
MongoDB	<code>mongosh</code>	Ingresar a la consola de MongoDB.
MongoDB	<code>use tienda_adso</code>	Crear o seleccionar la base de datos.
MongoDB	<code>db.createCollection()</code>	Crear colecciones.
MongoDB	<code>insertMany()</code>	Insertar varios documentos.
MongoDB	<code>find()</code>	Consultar documentos.
MongoDB	<code>findOne()</code>	Consultar un solo documento.
MongoDB	<code>\$gt, \$lte, \$eq</code>	Aplicar filtros.
MongoDB	<code>sort()</code>	Ordenar resultados.
MongoDB	<code>aggregate()</code>	Realizar consultas más elaboradas.
MongoDB	<code>\$lookup</code>	Relacionar colecciones en una agregación.
MongoDB	<code>\$unwind</code>	Separar elementos de un arreglo.
MongoDB	<code>\$project</code>	Seleccionar campos de salida.
MongoDB	<code>\$group</code>	Agrupar resultados.

Categoría	Comando u operación	Uso en el taller
MongoDB	<code>updateOne()</code>	Actualizar un documento.
MongoDB	<code>updateMany()</code>	Actualizar varios documentos.
MongoDB	<code>deleteOne()</code>	Eliminar un documento.
MongoDB	<code>deleteMany()</code>	Eliminar varios documentos.
MongoDB	<code>drop()</code>	Eliminar una colección.
MongoDB	<code>dropDatabase()</code>	Eliminar una base de datos.
Administración	<code>mongodump</code>	Crear copia de seguridad.
Administración	<code>mongorestore</code>	Restaurar copia de seguridad.

MongoDB documenta `mongodump` y `mongorestore` como herramientas para respaldo y restauración de bases de datos.

12. Desarrollo guiado del taller

Momento 1. Crear carpeta del proyecto

Crear una carpeta para organizar el taller:

```
mkdir taller_mongodb_adso
cd taller_mongodb_adso
```

Dentro de esta carpeta se guardarán:

- `docker-compose.yml`
- Carpeta backup
- Archivo de comandos `script_mongodb.js`
- Capturas de pantalla
- Documento de reflexión individual

Crear carpeta para respaldo:

```
mkdir backup
```

13. Docker Compose para tener el servicio de MongoDB

13.1 ¿Qué es Docker Compose?

Docker Compose permite definir uno o varios servicios en un archivo llamado normalmente `docker-compose.yml` o `compose.yml`.

En este taller se usará para crear el servicio de MongoDB sin instalar MongoDB directamente en el computador.

Compose permite configurar aspectos como imagen, puertos, volúmenes, redes y variables de entorno desde un solo archivo.

13.2 Crear archivo `docker-compose.yml`

En la carpeta del taller, crear un archivo llamado:

`docker-compose.yml`

Agregar el siguiente contenido:

```
services:
  mongodb:
    image: mongo:8.0
    container_name: mongodb_tienda_adso
    restart: unless-stopped
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: Admin12345
    volumes:
      - mongo_data_adso:/data/db
      - ./backup:/backup
```

```
volumes:
  mongo_data_adso:
```

13.3 Explicación del archivo

Elemento	Explicación
services	Define los servicios que se van a levantar.
mongodb	Nombre del servicio.
image: mongo:8.0	Indica que se usará la imagen oficial de MongoDB.
container_name	Nombre personalizado del contenedor.

Elemento	Explicación
<code>restart: unless-stopped</code>	Permite reiniciar el servicio si se detiene, excepto si el usuario lo detiene manualmente.
<code>ports</code>	Expone el puerto 27017 del contenedor al equipo local.
<code>environment</code>	Define usuario y contraseña iniciales.
<code>volumes</code>	Permite conservar datos y guardar respaldos.
<code>mongo_data_adso</code>	Volumen persistente para los datos de MongoDB.
<code>./backup:/backup</code>	Carpeta local para guardar y recuperar copias de seguridad.

La imagen oficial de MongoDB en Docker Hub indica que el servidor MongoDB escucha normalmente en el puerto 27017 y permite configurar usuario y contraseña inicial mediante variables de entorno.

14. Levantar MongoDB con Docker Compose

Paso 1. Verificar Docker

```
docker --version
docker compose version
```

Paso 2. Levantar el servicio

```
docker compose up -d
```

Paso 3. Verificar contenedor activo

```
docker compose ps
```

También se puede usar:

```
docker ps
```

Paso 4. Revisar logs

```
docker compose logs mongodb
```

Paso 5. Ingresar a MongoDB con `mongosh`

```
docker exec -it mongodb_tienda_adso mongosh -u admin -p Admin12345 --authenticationDatabase admin
```

Si el ingreso fue correcto, aparecerá la consola de MongoDB.

15. Crear y usar la base de datos

Paso 6. Crear o seleccionar la base de datos

```
use tienda_adso
```

En MongoDB, la base de datos se crea realmente cuando se inserta el primer documento.

Paso 7. Ver la base de datos actual

```
db
```

Paso 8. Crear colecciones

```
db.createCollection("clientes")
db.createCollection("productos")
db.createCollection("pedidos")
```

Paso 9. Ver colecciones creadas

```
show collections
```

16. Insertar documentos

Los datos se toman del taller base de MySQL, donde se insertan clientes, productos, pedidos y detalle de pedido para la tienda tienda_adso.

Momento 2. Insertar clientes

```
db.clientes.insertMany([
  {
    _id: 1,
    tipo_documento: "CC",
    numero_documento: "1001234567",
    nombre: "Ana Pérez",
    correo: "ana.perez@correo.com",
    telefono: "3001112233",
    fecha_registro: new Date()
  },
  {
    _id: 2,
```

```

    tipo_documento: "CC",
    numero_documento: "1002345678",
    nombre: "Carlos Ruiz",
    correo: "carlos.ruiz@correo.com",
    telefono: "3102223344",
    fecha_registro: new Date()
  },
  {
    _id: 3,
    tipo_documento: "TI",
    numero_documento: "1003456789",
    nombre: "María Gómez",
    correo: "maria.gomez@correo.com",
    telefono: "3203334455",
    fecha_registro: new Date()
  },
  {
    _id: 4,
    tipo_documento: "CC",
    numero_documento: "1004567890",
    nombre: "Laura Torres",
    correo: "laura.torres@correo.com",
    telefono: "3014445566",
    fecha_registro: new Date()
  }
])

```

Verificar:

```
db.clientes.find()
```

Consultar con formato más legible:

```
db.clientes.find().pretty()
```

Momento 3. Insertar productos

```

db.productos.insertMany([
  {
    _id: 1,
    codigo_producto: "PROD001",
    nombre_producto: "Mouse inalámbrico",
    descripcion: "Mouse óptico USB inalámbrico",
    precio: 45000,
    stock: 20,
    activo: true
  },
  {
    _id: 2,

```

```

    codigo_producto: "PROD002",
    nombre_producto: "Teclado mecánico",
    descripcion: "Teclado mecánico retroiluminado",
    precio: 180000,
    stock: 10,
    activo: true
  },
  {
    _id: 3,
    codigo_producto: "PROD003",
    nombre_producto: "Audífonos Bluetooth",
    descripcion: "Audífonos inalámbricos con micrófono",
    precio: 95000,
    stock: 15,
    activo: true
  },
  {
    _id: 4,
    codigo_producto: "PROD004",
    nombre_producto: "Base refrigerante",
    descripcion: "Base para computador portátil",
    precio: 70000,
    stock: 8,
    activo: true
  }
]
])

```

Verificar:

```
db.productos.find().pretty()
```

17. Consultas básicas

Momento 4. Consultar todos los clientes

```
db.clientes.find()
```

Momento 5. Consultar productos disponibles

```

db.productos.find(
  {},
  {
    nombre_producto: 1,
    precio: 1,
    stock: 1,
    _id: 0
  }
)

```


Explicación:

El primer {} indica que no se aplica filtro.

El segundo objeto indica qué campos se quieren mostrar.

_id: 0 oculta el identificador.

Momento 6. Consultar productos con precio mayor a 80000

```
db.productos.find(
  { precio: { $gt: 80000 } },
  { nombre_producto: 1, precio: 1, _id: 0 }
)
```

Momento 7. Consultar clientes ordenados alfabéticamente

```
db.clientes.find(
  {},
  { nombre: 1, correo: 1, telefono: 1, _id: 0 }
).sort({ nombre: 1 })
```

18. Insertar pedidos con detalle embebido

En el taller de MySQL, el detalle del pedido se maneja en una tabla separada llamada detalle_pedido. En MongoDB se trabajará como un arreglo de objetos dentro de cada pedido.

Momento 8. Insertar pedidos

```
db.pedidos.insertMany([
  {
    _id: 1,
    id_cliente: 1,
    fecha_pedido: new Date(),
    estado: "CREADO",
    total: 225000,
    detalle_pedido: [
      {
        id_producto: 1,
        cantidad: 1,
        precio_unitario: 45000
      },
      {
        id_producto: 2,
        cantidad: 1,
        precio_unitario: 180000
      }
    ]
  }
],
{
  w: 1,
  wtimeout: 5000,
  journal: true,
  fsync: true
})
```

```

{
  _id: 2,
  id_cliente: 2,
  fecha_pedido: new Date(),
  estado: "PAGADO",
  total: 95000,
  detalle_pedido: [
    {
      id_producto: 3,
      cantidad: 1,
      precio_unitario: 95000
    }
  ]
},
{
  _id: 3,
  id_cliente: 3,
  fecha_pedido: new Date(),
  estado: "CREADO",
  total: 180000,
  detalle_pedido: [
    {
      id_producto: 2,
      cantidad: 1,
      precio_unitario: 180000
    }
  ]
}
]
])

```

Verificar:

```
db.pedidos.find().pretty()
```

19. Comprender las relaciones en MongoDB

Momento 9. Lectura guiada del modelo

Observa este documento:

```
db.pedidos.findOne({ _id: 1 })
```

Interpretación:

El pedido número 1 pertenece al cliente con `_id: 1`.

```
id_cliente: 1
```

El pedido tiene dos productos en el arreglo detalle_pedido:

```
detalle_pedido: [  
  {  
    id_producto: 1,  
    cantidad: 1,  
    precio_unitario: 45000  
  },  
  {  
    id_producto: 2,  
    cantidad: 1,  
    precio_unitario: 180000  
  }  
]
```

Esto significa que el pedido 1 incluye:

- Producto 1: Mouse inalámbrico.
- Producto 2: Teclado mecánico.

La información completa puede consultarse usando referencias entre colecciones o agregaciones.

20. Consultas relacionadas con agregación

Momento 10. Consultar pedidos con nombre del cliente

```
db.pedidos.aggregate([  
  {  
    $lookup: {  
      from: "clientes",  
      localField: "id_cliente",  
      foreignField: "_id",  
      as: "cliente"  
    }  
  },  
  {  
    $unwind: "$cliente"  
  },  
  {  
    $project: {  
      _id: 0,  
      id_pedido: "$_id",  
      cliente: "$cliente.nombre",  
      fecha_pedido: 1,  
      estado: 1,  
      total: 1  
    }  
  }  
])
```

```

    }
  }
])

```

Explicación:

Esta consulta une la colección pedidos con la colección clientes.

En MySQL se usaba JOIN; en MongoDB se puede usar \$lookup dentro de una agregación.

Momento 11. Consultar productos incluidos en cada pedido

```

db.pedidos.aggregate([
  {
    $unwind: "$detalle_pedido"
  },
  {
    $lookup: {
      from: "productos",
      localField: "detalle_pedido.id_producto",
      foreignField: "_id",
      as: "producto"
    }
  },
  {
    $unwind: "$producto"
  },
  {
    $project: {
      _id: 0,
      id_pedido: "$_id",
      nombre_producto: "$producto.nombre_producto",
      cantidad: "$detalle_pedido.cantidad",
      precio_unitario: "$detalle_pedido.precio_unitario"
    }
  }
])

```

Explicación:

Esta consulta permite recorrer el arreglo detalle_pedido, buscar el producto relacionado y mostrar su nombre.

Momento 12. Consultar pedido completo con cliente y productos

```

db.pedidos.aggregate([
  {

```

```

    $lookup: {
      from: "clientes",
      localField: "id_cliente",
      foreignField: "_id",
      as: "cliente"
    }
  },
  {
    $unwind: "$cliente"
  },
  {
    $unwind: "$detalle_pedido"
  },
  {
    $lookup: {
      from: "productos",
      localField: "detalle_pedido.id_producto",
      foreignField: "_id",
      as: "producto"
    }
  },
  {
    $unwind: "$producto"
  },
  {
    $project: {
      _id: 0,
      id_pedido: "$_id",
      cliente: "$cliente.nombre",
      nombre_producto: "$producto.nombre_producto",
      cantidad: "$detalle_pedido.cantidad",
      precio_unitario: "$detalle_pedido.precio_unitario",
      subtotal: {
        $multiply: [
          "$detalle_pedido.cantidad",
          "$detalle_pedido.precio_unitario"
        ]
      },
      estado: "$estado"
    }
  },
  {
    $sort: {
      id_pedido: 1
    }
  }
])

```

Esta consulta responde:

- ¿Quién realizó el pedido?
- ¿Qué productos compró?
- ¿Cuántas unidades compró?
- ¿Cuál fue el precio unitario?
- ¿Cuál fue el subtotal?
- ¿En qué estado está el pedido?

Momento 13. Consultar total calculado por pedido

```
db.pedidos.aggregate([
  {
    $unwind: "$detalle_pedido"
  },
  {
    $group: {
      _id: "$_id",
      id_cliente: { $first: "$id_cliente" },
      total_guardado: { $first: "$total" },
      total_calculado: {
        $sum: {
          $multiply: [
            "$detalle_pedido.cantidad",
            "$detalle_pedido.precio_unitario"
          ]
        }
      }
    }
  },
  {
    $lookup: {
      from: "clientes",
      localField: "id_cliente",
      foreignField: "_id",
      as: "cliente"
    }
  },
  {
    $unwind: "$cliente"
  },
  {
    $project: {
      _id: 0,
      id_pedido: "$_id",
      cliente: "$cliente.nombre",
      total_guardado: 1,
      total_calculado: 1
    }
  }
])
```

```
}  
])
```

Pregunta para reflexionar:

¿Por qué el total calculado desde detalle_pedido debe coincidir con el total guardado en el documento pedidos?

21. Modificar estructuras en MongoDB

En MySQL se usa ALTER TABLE para modificar una estructura. En MongoDB no se modifica una tabla, porque las colecciones son flexibles. Se pueden agregar nuevos campos a documentos existentes usando \$set.

Momento 14. Agregar campo dirección a un cliente

```
db.clientes.updateOne(  
  { _id: 1 },  
  {  
    $set: {  
      direccion: "Calle 10 # 20-30"  
    }  
  }  
)
```

Verificar:

```
db.clientes.findOne({ _id: 1 })
```

Momento 15. Agregar campo categoría a todos los productos

```
db.productos.updateMany(  
  {},  
  {  
    $set: {  
      categoria: "Accesorios tecnológicos"  
    }  
  }  
)
```

Consultar:

```
db.productos.find(  
  {},  
  {  
    nombre_producto: 1,  
    categoria: 1,  
    _id: 0  
  })
```

```
) }  
)
```

22. Actualizar documentos

Momento 16. Cambiar estado de un pedido

```
db.pedidos.updateOne(  
  { _id: 1 },  
  {  
    $set: {  
      estado: "PAGADO"  
    }  
  }  
)
```

Verificar:

```
db.pedidos.findOne({ _id: 1 })
```

Momento 17. Cambiar precio de un producto

```
db.productos.updateOne(  
  { _id: 1 },  
  {  
    $set: {  
      precio: 50000  
    }  
  }  
)
```

Verificar:

```
db.productos.findOne({ _id: 1 })
```

Pregunta para reflexionar:

Si cambia el precio actual del producto en la colección productos, ¿debería cambiar automáticamente el precio de pedidos anteriores?

Respuesta esperada:

No necesariamente. Por eso en el arreglo detalle_pedido se guarda precio_unitario, porque representa el precio aplicado en el momento de la compra.

23. Eliminar documentos de forma controlada

Momento 18. Eliminar un producto del detalle de un pedido

Antes de eliminar:

```
db.pedidos.findOne({ _id: 3 })
```

Eliminar del arreglo detalle_pedido el producto con id_producto: 2 en el pedido 3:

```
db.pedidos.updateOne(
  { _id: 3 },
  {
    $pull: {
      detalle_pedido: {
        id_producto: 2
      }
    }
  }
)
```

Verificar:

```
db.pedidos.findOne({ _id: 3 })
```

Momento 19. Revisar antes de eliminar un cliente

Antes de eliminar un cliente, se debe revisar si tiene pedidos asociados.

```
db.pedidos.find({ id_cliente: 1 })
```

Pregunta:

¿Por qué no sería correcto eliminar un cliente que tiene pedidos asociados sin revisar primero la información?

Explicación:

En MongoDB, si se ejecuta `deleteOne` sobre un cliente, MongoDB no necesariamente impedirá la eliminación aunque existan pedidos relacionados. Por eso, el aprendiz debe comprender que la integridad de los datos debe ser cuidada desde el diseño y desde la lógica de la aplicación.

No ejecutar todavía:

```
db.clientes.deleteOne({ _id: 1 })
```

24. Vaciar una colección

Momento 20. Precaución antes de usar `deleteMany`

`deleteMany({})` elimina todos los documentos de una colección.

Antes de usarlo, se debe hacer copia de seguridad o estar completamente seguro.

Ejemplo:

```
db.pedidos.deleteMany({})
```

Para este taller, se realizará primero copia de seguridad.

25. Copia de seguridad de la base de datos

Momento 21. Salir de MongoDB

`exit`

Momento 22. Crear copia de seguridad con `mongodump`

Desde CMD, PowerShell o terminal, ejecutar:

```
docker exec mongodb_tienda_adso mongodump \  
  --username admin \  
  --password Admin12345 \  
  --authenticationDatabase admin \  
  --db tienda_adso \  
  --archive=/backup/tienda_adso_backup.archive
```

En Windows PowerShell, si el salto de línea genera error, usar el comando en una sola línea:

```
docker exec mongodb_tienda_adso mongodump --username admin --password Admin12345 --authenticationDatabase admin --db tienda_adso --archive=/backup/tienda_adso_backup.archive
```

Momento 23. Verificar respaldo

Verificar en la carpeta local backup que exista el archivo:

`tienda_adso_backup.archive`

El aprendiz debe tomar captura de:

- Comando ejecutado.
- Archivo `tienda_adso_backup.archive` creado.

- Carpeta donde quedó guardado.

26. Borrado controlado después del respaldo

Momento 24. Ingresar nuevamente a MongoDB

```
docker exec -it mongodb_tienda_adso mongosh -u admin -p Admin12345 --authenticationDatabase admin
```

Seleccionar base:

```
use tienda_adso
```

Momento 25. Vaciar colección pedidos

```
db.pedidos.deleteMany({})
```

Verificar:

```
db.pedidos.find()
```

Momento 26. Eliminar una colección

```
db.productos.drop()
```

Verificar:

```
show collections
```

Momento 27. Eliminar la base de datos

```
db.dropDatabase()
```

Verificar:

```
show dbs
```

27. Restauración de la base de datos

Momento 28. Salir de MongoDB

```
exit
```

Momento 29. Restaurar desde el respaldo

Desde CMD, PowerShell o terminal:

```
docker exec mongodb_tienda_adso mongorestore \  
  --username admin \  
  --password Admin12345 \  
  --authenticationDatabase admin \  
  --archive=/backup/tienda_adso_backup.archive \  
  --nsInclude="tienda_adso.*" \  
  --drop
```

En Windows PowerShell, si el salto de línea genera error, usar:

```
docker exec mongodb_tienda_adso mongorestore --username admin --password  
Admin12345 --authenticationDatabase admin --archive=/backup/tienda_adso_b  
ackup.archive --nsInclude="tienda_adso.*" --drop
```

Momento 30. Verificar restauración

Ingresar nuevamente:

```
docker exec -it mongodb_tienda_adso mongosh -u admin -p Admin12345 --auth  
enticationDatabase admin
```

Seleccionar base:

```
use tienda_adso
```

Ver colecciones:

```
show collections
```

Consultar datos:

```
db.clientes.find().pretty()  
db.productos.find().pretty()  
db.pedidos.find().pretty()
```

28. Producto final del aprendiz

Cada aprendiz debe entregar una carpeta digital con:

1. Archivo docker-compose.yml.
2. Script completo de comandos MongoDB, llamado script_mongodb.js.
3. Captura de Docker funcionando.
4. Captura de docker compose ps con MongoDB activo.
5. Captura del ingreso a mongosh.
6. Captura de la base de datos tienda_adso.
7. Captura de las colecciones creadas.
8. Captura de documentos insertados en clientes.

9. Captura de documentos insertados en productos.
10. Captura de documentos insertados en pedidos.
11. Captura de consultas básicas con find.
12. Captura de filtros con operadores como \$gt o \$lte.
13. Captura de ordenamiento con sort.
14. Captura de consulta con agregación y \$lookup.
15. Captura de actualización con updateOne.
16. Captura de actualización con updateMany.
17. Captura de eliminación controlada con \$pull, deleteMany o drop.
18. Captura de copia de seguridad con mongodump.
19. Archivo tienda_adso_backup.archive.
20. Captura del borrado de colección o base de datos.
21. Captura de restauración exitosa con mongorestore.
22. Reflexión individual.
23. Video individual explicando el paso a paso realizado.

29. Reflexión individual

Pregunta	Respuesta
¿Qué entendí sobre una base de datos documental?	
¿Qué es una colección en MongoDB?	
¿Qué es un documento?	
¿Qué es un objeto JSON?	
¿Qué diferencia encontré entre una tabla de MySQL y una colección de MongoDB?	
¿Qué diferencia encontré entre una fila de MySQL y un documento de MongoDB?	
¿Para qué sirve el campo _id?	
¿Cómo se representó el detalle de pedido en MongoDB?	
¿Qué consulta me ayudó más a entender el modelo documental?	
¿Qué error cometí durante la práctica y cómo lo solucioné?	
¿Por qué es importante hacer copia de seguridad antes de borrar datos?	
¿Qué aprendí al restaurar la base de	

Pregunta	Respuesta
datos?	
¿Cómo se relaciona este taller con el modelado de datos?	

30. Video individual de explicación del paso a paso

Cada aprendiz realizará un video individual explicando el proceso realizado durante el taller.

Duración sugerida

Entre 5 y 8 minutos.

Condiciones del video

- Debe grabarse la pantalla.
- Debe escucharse la voz del aprendiz.
- El aprendiz debe explicar, no solo mostrar comandos.
- El video debe evidenciar que el aprendiz comprende el proceso.
- No se aceptará únicamente música de fondo o capturas sin explicación.
- El video debe entregarse mediante enlace o archivo según indicación del instructor.

Estructura sugerida del video

Momento	Tiempo sugerido	Contenido
Presentación	20 segundos	Nombre del aprendiz y objetivo de la práctica.
Docker Compose	60 segundos	Explicación del archivo <code>docker-compose.yml</code> y cómo se levantó MongoDB.
Base de datos	50 segundos	Explicación de la base <code>tienda_adso</code> y sus colecciones.
JSON y documentos	60 segundos	Explicación de qué es un objeto JSON y cómo se insertaron documentos.
Consultas	90 segundos	Mostrar consultas básicas, filtros y una

Momento	Tiempo sugerido	Contenido
		agregación con \$lookup.
Actualización y borrado	60 segundos	Explicar updateOne, updateMany, \$pull o deleteMany.
Backup y restauración	70 segundos	Mostrar mongodump, borrado controlado y mongorestore.
Aprendizaje final	30 segundos	Conclusión personal y dificultad superada.

Guion sugerido

Mi nombre es _____. En este taller construí una base de datos documental llamada tienda_adso usando MongoDB y Docker Compose.

Primero creé el archivo docker-compose.yml, donde configuré el servicio de MongoDB, el puerto 27017, el usuario administrador, la contraseña y los volúmenes para conservar los datos y guardar el respaldo.

Luego ingresé a MongoDB usando mongosh y creé la base de datos tienda_adso. Allí trabajé con las colecciones clientes, productos y pedidos.

Comprendí que en MongoDB los datos se almacenan en documentos parecidos a objetos JSON. Un objeto JSON usa pares clave-valor, por ejemplo nombre, correo, precio, stock o estado.

En la colección pedidos, el detalle del pedido se guardó como un arreglo de objetos dentro del mismo documento. Esto me permitió entender cómo MongoDB puede representar información relacionada de una forma diferente a MySQL.

También realicé consultas con find, filtros, ordenamiento y agregaciones con \$lookup. Después actualicé documentos, eliminé información de forma controlada, hice una copia de seguridad con mongodump, borré la base de datos y la restauré con mongorestore.

La principal dificultad que tuve fue _____, y la solucioné _____. Mi aprendizaje principal fue que MongoDB permite organizar datos de forma flexible, pero también exige responsabilidad al diseñar, consultar, respaldar y restaurar la información.

31. Lista de chequeo

N.º	Criterio de verificación	Cumple	En proceso	No cumple	Observaciones
1	Verifica correctamente Docker y Docker Compose.				
2	Crea la carpeta del taller.				
3	Crea el archivo docker-compose.yml.				
4	Explica los elementos principales del archivo Compose.				
5	Levanta MongoDB con docker compose up -d.				
6	Verifica el contenedor con docker compose ps.				
7	Ingresa correctamente a MongoDB con mongosh.				
8	Crea o selecciona la base de datos tienda_adso.				
9	Crea las colecciones clientes, productos y pedidos.				
10	Explica qué es un objeto JSON.				
11	Explica qué es un documento en MongoDB.				
12	Inserta documentos en la colección clientes.				
13	Inserta documentos en la colección productos.				
14	Inserta documentos en la colección pedidos.				
15	Usa arreglos para representar detalle_pedido.				
16	Consulta documentos con find.				
17	Aplica filtros con operadores como \$gt o \$lte.				

N.º	Criterio de verificación	Cumple	En proceso	No cumple	Observaciones
18	Ordena resultados con sort.				
19	Realiza una consulta con agregación y \$lookup.				
20	Actualiza documentos con updateOne.				
21	Actualiza varios documentos con updateMany.				
22	Elimina información de forma controlada.				
23	Realiza copia de seguridad con mongodump.				
24	Borra colección o base de datos después del respaldo.				
25	Restaura correctamente con mongorestore.				
26	Verifica que los datos restaurados estén completos.				
27	Entrega evidencias organizadas.				
28	Entrega el archivo de respaldo.				
29	Entrega reflexión individual.				
30	Entrega video individual explicando el paso a paso.				

32. Rúbrica de evaluación

Criterio	Nivel alto	Nivel medio	Nivel bajo
Comprensión de MongoDB	Explica claramente base de datos, colección, documento, campo, _id, JSON y BSON.	Reconoce algunos conceptos, pero confunde elementos.	No diferencia los conceptos básicos.
Uso de Docker Compose	Crea y explica correctamente el	Levanta el servicio, pero explica	No logra levantar MongoDB o no

Criterio	Nivel alto	Nivel medio	Nivel bajo
	archivo docker-compose.yml.	parcialmente la configuración.	comprende el archivo.
Inserción de documentos	Inserta correctamente clientes, productos y pedidos usando objetos JSON.	Inserta documentos, pero con errores menores de estructura.	No logra insertar los datos requeridos.
Consultas básicas	Consulta, filtra y ordena documentos correctamente.	Realiza consultas simples, pero requiere apoyo en filtros.	No logra consultar la información.
Consultas relacionadas	Usa agregaciones y \$lookup para relacionar clientes, pedidos y productos.	Realiza agregaciones con apoyo.	No logra relacionar la información.
Actualización y eliminación	Actualiza y elimina información de forma controlada y justificada.	Ejecuta comandos, pero con poca explicación.	Borra datos sin control o no logra ejecutar las operaciones.
Backup y restauración	Realiza respaldo, borrado y restauración comprobada.	Realiza parte del proceso, pero requiere apoyo.	No evidencia respaldo o restauración.
Video individual	Explica paso a paso con claridad, evidencia comandos y demuestra comprensión.	Presenta el video, pero la explicación es parcial.	El video no evidencia el proceso o no explica lo realizado.
Reflexión individual	Responde con análisis propio y relaciona el taller con modelado de datos.	Responde de forma básica.	No entrega reflexión o responde sin coherencia.

33. Errores comunes y solución

Error común	Posible causa	Solución
Docker no reconoce el comando	Docker no está instalado o no está abierto.	Abrir Docker Desktop o verificar instalación.
El contenedor no inicia	Puerto ocupado o error en <code>docker-compose.yml</code> .	Revisar logs con <code>docker compose logs mongodb</code> .
Error en puerto 27017	Otro MongoDB está usando el puerto.	Cambiar el puerto local, por ejemplo <code>"27018:27017"</code> .
No permite ingresar con usuario y contraseña	Usuario, contraseña o <code>authenticationDatabase</code> incorrectos.	Revisar variables de entorno y comando de conexión.
Error de sintaxis JSON	Comas, llaves o comillas mal ubicadas.	Revisar estructura del documento.
No aparece la base de datos	No se han insertado documentos.	Insertar al menos un documento.
Se borraron datos por error	Se ejecutó <code>deleteMany</code> , <code>drop</code> o <code>dropDatabase</code> .	Restaurar desde respaldo.
No aparece el archivo de backup	Carpeta backup no fue montada correctamente.	Revisar volumen <code>./backup:/backup</code> en Compose.
La restauración no funciona	Ruta incorrecta del archivo o base mal especificada.	Verificar nombre del archivo y comando <code>mongorestore</code> .
El video no evidencia el proceso	Solo se muestran capturas sin explicación.	Grabar pantalla y voz explicando paso a paso.

34. Cierre pedagógico

Este taller permite comprender que MongoDB organiza la información de manera diferente a una base de datos relacional. En lugar de trabajar principalmente con tablas y claves foráneas, MongoDB trabaja con colecciones y documentos similares a objetos JSON.

El aprendiz debe comprender que MongoDB ofrece flexibilidad, pero esa flexibilidad exige mayor responsabilidad al momento de diseñar los documentos, representar relaciones, evitar datos inconsistentes, realizar copias de seguridad y restaurar la información cuando sea necesario.

El desarrollo de este taller conecta directamente con el modelado de datos, porque permite comparar dos formas de representar un mismo problema:

- En MySQL: tablas relacionadas.
- En MongoDB: colecciones, documentos, referencias y documentos embebidos.

35. Fuentes confiables recomendadas

1. **Docker Docs — Docker Compose**
Fuente oficial para comprender cómo definir y ejecutar servicios mediante archivos Compose, incluyendo servicios, redes y volúmenes.
2. **Docker Docs — How Compose Works**
Fuente oficial para explicar cómo Compose crea servicios, redes y volúmenes al ejecutar `docker compose up`.
3. **Docker Hub — Mongo Official Image**
Fuente oficial para usar la imagen de MongoDB en Docker, configurar variables de entorno y exponer el puerto 27017.
4. **MongoDB Docs — Documents**
Fuente oficial para explicar documentos BSON, estructura similar a JSON, campos y valores.
5. **MongoDB Docs — CRUD Operations**
Fuente oficial para operaciones de creación, lectura, actualización y eliminación de documentos.
6. **MongoDB Docs — mongodump**
Fuente oficial para respaldo de bases de datos con MongoDB Database Tools.

7. **MongoDB Docs — mongorestore**

Fuente oficial para restaurar archivos BSON generados desde mongodump.